

AI 활용 기술 문서 — Rewind Survivors

AI를 "코드를 받아쓰는 도구"가 아니라 **규칙·역할·지식이 파일로 고정된 개발 시스템**으로 구성했다. 프로젝트 규칙 단일 원본, 역할별 전문 에이전트 5종, 오케스트레이터 스킴, **실제로 밝은 버그를 자산화한 지식 베이스 16건**이 모두 저장소에 커밋되어 있다.

저장소	https://github.com/rg7155/26UnityProject
웹 플레이	https://rg7155.github.io/26UnityProject/
플레이 영상	https://youtu.be/oo_amdaKNRk
규모	C# 106개 파일 / 약 10,100줄 · 커밋 90개
AI 자산	에이전트 5 · 스킴 2 · 지식 베이스 16건

1. 사용 도구와 에셋 출처

도구	용도
Claude Code (Opus)	설계·구현·리뷰 전반. 하네스 파이프라인 실행
Codex	동일 규칙을 공유하는 보조 실행기
ChatGPT	2D 스프라이트 생성 (플레이어·적 3종·보스·무기)
Suno AI	BGM 및 효과음 5종 생성

외부 에셋·오픈소스 출처

에셋	출처 / 라이선스
스프라이트 전체	ChatGPT 생성 — 외부 저작물 없음
BGM · 효과음 전체	Suno AI 생성 — 외부 저작물 없음
UI 전체	코드로 절차 생성(9-slice 런타임 제작). 외부 이미지 리소스 0개
NotoSansKR	Google Fonts, SIL Open Font License 1.1
LiberationSans	Unity TextMesh Pro 기본 동봉 (SIL OFL)
엔진·패키지	Unity 6, URP, Burst, Collections, Input System (Unity Companion License)

2. AI 하네스 구조

도구마다 설정이 흩어지면 규칙이 서로 어긋난다. **규칙과 역할의 원본을 각각 한 곳에 두고, 도구별 파일은 얇은 어댑터로만 두었다.**

경로	역할
AGENTS.md	프로젝트 규칙 단일 원본 — 코딩 스타일·기술 결정·금지 사항·트리거
CLAUDE.md	@AGENTS.md 를 import 만 하는 어댑터

경로	역할
.codex/agents/	Codex TOML 어댑터
.agents/roles/	역할 지시문 원본 5종 (Claude 사본은 스크립트로 동기화)
.agents/skills/	unity-dev(오케스트레이터) · ui-kit(코드기반 UGUI 규격)
.claude/knowledge/	지식 베이스 16건 — bug-patterns / lessons-learned / workflows

파이프라인

요청 → **game-designer**(무엇을·왜) → *사람이 방향 확정* → **Planner**(단계별 계획 → plan.md) → **game-coder**(로직) · **game-ui-artist**(UGUI) → **game-reviewer**(옵트인) → *사람이 플레이 검증*

설계 의도 세 가지.

- **역할 분리** — 단계마다 맥락이 좁아져 지시가 정확해진다. 한 에이전트에 전부 맡기면 요청하지 않은 기능이 섞여 들어온다.
- **파일 분담 강제** — 계획서가 에이전트별 담당 파일을 지정해 **교집합을 0**으로 만든다. 같은 파일 충돌을 구조적으로 차단한다.
- **리뷰는 기본 스킵** — UI는 스크린샷으로, 로직은 컴파일·플레이로 검증된다. 세이브·되감기처럼 **눈으로 검증이 안 되는 변경**에만 사람이 호출한다.

3. AI에게 준 주요 지시 (실제 발췌)

코딩 원칙 — LLM의 과잉 구현을 막기 위해 AGENTS.md 최상단에 배치

```
## 2. 단순함 우선
- 요청된 것만 구현. 추측성 기능 없음.
- 한 번만 쓰이는 코드에 추상화 없음.
- 200줄로 쓸 수 있는 걸 50줄로 쓸 수 있다면 다시 씀.
```

```
## 3. 외과적 변경
- 필요한 것만 건드림. 인접 코드·주석·포맷 개선 금지.
- 관련 없는 dead code 발견 시 언급만, 삭제 금지.
```

프로젝트 제약 — AI가 매번 "더 나은 방식"을 제안해 구조가 흔들리는 것을 막기 위해 결정과 이유를 표로 고정

```
| Input System | New Input System — Polling 방식 | Action-based 사용 안 함 |
| 리소스 로딩 | Resources.Load | Addressables 미사용. 포트폴리오 규모에 충분 |
```

```
### 금지 사항
- DI 프레임워크(Zenject, VContainer) 제안 금지 — 규모 초과
- 과도한 인터페이스/제네릭 추상화 금지
```

주석 규약 — WHAT 금지, WHY만. 그 결과 산출물의 주석은 전부 "왜"를 설명한다.

```
브라우저는 첫 제스처 전까지 AudioContext 를 잠근다. 잠긴 채 Play 하면
재생 헤드만 흘러가 도입부가 유실되고, isPlaying 은 true 라 감지도 불가능하다.
그래서 첫 입력까지 기다린다.
```

착수 전 지식 베이스 검색 — 버그 수정·기능 구현 전 `.claude/knowledge/` 를 `symptom-tags` 기준으로 검색하고, 재발 위험이 있는 문제를 새로 해결하면 엔트리로 저장하도록 트리거를 배선했다.

단계별 구현 강제 — 계획서에 단계마다 "여기서 멈춰도 동작하는가" 를 명시하게 했다. 그 결과 보스 시스템은 Stage 6이 최소 출하선이 되고 7~10은 컷 가능한 구조로 설계되었다.

4. 사례 — 보스 시스템 (파이프라인 완주)

커밋 3개(29ed438·ac389f0·5ce5362), 신규 C# 10개 파일 + 수정 20여 개. `game-designer` → 사람 확정 → `Planner` → `game-coder` ×6(Stage 단위) → `game-ui-artist` ×2 → 플레이 검증.

AI가 내린 설계 판단 — 되감기 스냅샷 배치 3안 비교

안	판정
EnemySnapshot 확장	1개체용 필드를 적 300×100프레임 전체에 부과
별도 링버퍼	인덱스·리셋이 2벌 → 어긋나면 재현성이 조용히 깨진다
FrameSnapshot에 프레임당 1개	○ 기존 WaveSnapshot 선례와 동일

일반 적 파이프라인 편입을 항목별로 판단 — `EnemyBase·SpatialHash`는 편입(무기 판정·되감기 재사용), GPU 인스턴싱과 Burst Job은 **의도적 제외**(개체별 색 제어 불가 / 잡몹에 밀리면 패턴 시작 위치가 흔들려 결정성 붕괴).

제약을 근거를 들어 깬 사례 — "EnemyBase 수정 금지" 제약이 있었으나, 되감기 캡처가 `EnemyMover·AttackCooldown`만 읽어 **보스 쿨다운이 유실**되는 문제를 발견. virtual 프로퍼티 추가가 기존 확장 패턴과 동일하고 hot path의 as 캐스트도 제거된다는 근거를 제시해 승인받았다.

기획이 정체성과 연결된 지점 — 보스 패턴을 고정 시퀀스로 두어 되감기가 "주사위 재굴림"이 아니라 "재도전" 이 되게 했다. 보스 체력도 함께 되감기므로 그 자체가 남용 방지 장치가 되어 별도 페널티가 필요 없다.

5. AI를 검증한 기록

AI 결과물을 그대로 받아쓰지 않았다는 증거로 **실패와 반박을 그대로 남긴다**.

AI가 틀렸던 것

사안	무엇이 틀렸나
보스 리쉬 거리	18로 산출했으나 가로 화면 전제 . 세로 화면의 반대각선은 11.5
프리팸 복제 안내	GPU 인스턴싱 적 프리팸엔 <code>SpriteRenderer·Rigidbody2D</code> 가 없다는 사실을 놓쳐, 복제한 보스가 안 보이거나 낙하
무기 존재 여부 조사	"Orbit-Lightning은 어디에도 참조되지 않는다"고 단정했으나 실제로는 데모 모드에 존재. 결론은 유효했으나 근거가 틀렸다

사람이 AI 판단을 뒤집은 것

사안	AI 제안	사람의 결정과 근거
2번째 보스 추가	"데이터만으로 가능해 ROI 최고"	취소. 신규 패턴 0인 리스킨이라 콘텐츠 깊이를 못 건드리고, 데이터 주도 증명은 이미 4회 했다
보스 등장 방식	런 중 등장만	BOSS RUSH 데모 추가. "심사위원이 3분을 생존할 리 없다"
페이지 2 체감	색 변화 + 쿨다운 단축	"색만 바뀐다" 지적 → 한 바퀴 22.6초 vs 전투 35~45초라 25% 단축이 표본에 안 잡힌다 는 계산으로 원인 규명 후 속도·피해 축 추가

6. 지식 베이스 — 버그를 자산으로

실제로 밝은 버그를 "증상 → 근본 원인 → 재인식 패턴" 형식으로 16건 추적했다. 착수 전 검색용이다.

jsonutility-value-type-needs-schema-version — 볼륨 float을 세이브에 추가했더니 구버전 세이브에서 0으로 남아 **전 게임이 무음**이 됐다. 참조 타입은 null이라는 신호가 남지만 **값 타입은 0.0f가 "미저장"과 "사용자가 음소거함"을 구분하지 못한다.** 필드 가드가 원리적으로 불가능해 SchemaVersion을 도입했다. 이 엔트리는 기존 엔트리의 **"값 타입은 안전하다"**는 서술이 **틀렸음을 드러냈고**, 해당 엔트리에 단서를 달아 상호 참조를 걸었다.

meshrenderer-sorting-order-hidden-in-inspector — 배경을 타일 쿼드로 교체한 뒤 보스 예고원이 사라졌다. sortingOrder가 z보다 우선하는데 배경 기본값 0 > 예고원 20이라 가려진 것. 판정과 화면 흔들림은 정상이라 로직 버그로 오인하기 쉬웠다. MeshRenderer는 이 필드를 **Inspector에 노출하지 않아** 코드로만 지정 가능하다.

7. 결과

규칙이 파일로 고정되어 세션이 바뀌어도 구조 결정이 흔들리지 않았고, 계획서와 KB가 남아 "왜 이렇게 만들었는지"를 추적할 수 있다. 데이터 주도 설계 덕에 무기·상점·퀘스트·웨이브는 **C# 0줄로** 콘텐츠를 추가한다.

AI가 만든 것은 코드지만, **무엇을 만들지 정하고 결과를 검증하고 틀린 판단을 뒤집는 일은 사람이 했다.** 5장이 그 기록이다.